

--- title: "Federação Zero-Trust — Segurança Avançada para Agentes A2A"  
subtitle: "Como agentes de plataformas diferentes se comunicam sem confiar em ninguém" author: "MMN\_IA Collective" version: "1.0.0" date: 2026-07-10 pattern: "MMN\_IA" --- \*\*Apostila 30 · Federação Zero-Trust\*\*  
\*Como aplicar os princípios de Zero-Trust à comunicação agent-to-agent, sem abrir mão da velocidade.\* \*\*Por MMN\_IA Collective · Academ'IA\*\*  
Nexus Affil'IA'te · 2026 \*\*Sobre esta apostila\*\* Zero-Trust ("nunca confiar, sempre verificar") é o padrão de segurança corporativa de 2020+. Mas a maioria das implementações é para \*\*humanos e serviços internos\*\*. E quando você tem \*\*agentes A2A\*\* (de plataformas diferentes), o desafio é maior: você não controla o outro agente. Esta apostila explica como aplicar Zero-Trust em uma rede federada de agentes. --- # Sumário \*\*PARTE I — FUNDAMENTOS\*\* 1. [O que é Zero-Trust](#cap1) 2. [Por que Zero-Trust é importante para A2A](#cap2) 3. [Os 6 pilares do Zero-Trust](#cap3) \*\*PARTE II — IMPLEMENTAÇÃO\*\* 4. [Identidade verificável (mTLS + DIDs)](#cap4) 5. [Autorização por skill (scope-based)](#cap5) 6. [Auditoria distribuída (audit logs)](#cap6) 7. [Rate limiting entre agentes](#cap7) \*\*PARTE III — CASOS PRÁTICOS\*\* 8. [Caso Nexus: 12 agentes em 4 plataformas](#cap8) 9. [Métricas de segurança](#cap9) 10. [Quando o Zero-Trust atrapalha](#cap10) Epílogo: [O futuro do trust entre agentes](#epilogo) Apêndice: [Checklist de implementação](#apendice) --- # Capítulo 1 — O que é Zero-Trust \*\*Zero-Trust\*\* é uma arquitetura de segurança que \*\*não confia automaticamente\*\* em nada — nem em usuários internos, nem em serviços, nem em redes "seguras". \*\*Princípio central\*\*: cada requisição é avaliada como se viesse de uma rede pública. Verificação contínua. \*\*Origem\*\*: conceito cunhado por John Kindervag (Forrester, 2010). Adotado por Google BeyondCorp (2014), NIST SP 800-207 (2020), e hoje padrão corporativo. \*\*Por que importa\*\*: 80% dos ataques envolvem movimento lateral (atacante entra em 1 máquina e se espalha). Zero-Trust reduz isso a < 5%. --- # Capítulo 2 — Por que Zero-Trust é importante para A2A Em comunicação A2A, você tem \*\*3 problemas adicionais\*\*: 1. \*\*Você não controla o outro agente\*\* — pode ser de qualquer vendor 2. \*\*Você não controla a rede\*\* — passa por internet pública 3. \*\*Você não controla o usuário final\*\* — pode ser qualquer um \*\*Riscos específicos A2A\*\*: - \*\*Agente malicioso\*\* finge ser legítimo e envia dados falsos - \*\*Man-in-the-middle\*\* intercepta comunicação - \*\*Replay attack\*\* repete request válido - \*\*Skill abuse\*\* chama skill com parâmetros maliciosos \*\*Solução\*\*: Zero-Trust + criptografia forte (mTLS, JWS) + autenticação por skill. --- # Capítulo 3 — Os 6 pilares do Zero-Trust 1. \*\*Identidade verificável\*\* — todo agente tem identidade criptográfica 2. \*\*Autenticação contínua\*\* — não basta logar uma vez 3. \*\*Autorização por recurso\*\* — cada skill tem seu próprio scope 4. \*\*Auditoria distribuída\*\* — todo request é logado 5. \*\*Criptografia everywhere\*\* — TLS 1.3 mínimo, mTLS em federation 6. \*\*Menor privilégio\*\* — agente só tem acesso ao que precisa \*\*Aplicado a A2A\*\*: | Pilar | Implementação A2A | |-----|-----| | Identidade | mTLS + DIDs (Decentralized Identifiers) | | Autenticação contínua | Token rotation a cada 1h | | Autorização | JWT com scope=skill:id | | Auditoria | OpenTelemetry traces + logs imutáveis | | Criptografia | TLS 1.3 + JWS para payloads sensíveis | | Menor privilégio | Cada agente só vê as skills declaradas no Agent Card | --- # Capítulo 4 — Identidade verificável (mTLS + DIDs) \*\*mTLS (Mutual TLS)\*\*: ambos os lados da conexão apresentam

certificado. Não só o servidor (como em HTTPS). ``python # Servidor A2A com mTLS from a2a import A2AServer import ssl context = ssl.create\_default\_context(ssl.Purpose.CLIENT\_AUTH) context.load\_cert\_chain('agent-cert.pem', 'agent-key.pem') context.load\_verify\_locations('ca-bundle.pem') server = A2AServer( agent\_card\_path=".well-known/agent.json", ssl\_context=context ) ``

**\*\*DIDs (Decentralized Identifiers)\*\*:** padrão W3C para identidade auto-soberana. Cada agente tem DID único (did:nexus:agent-abc123). ``json { "id": "did:nexus:agent-abc123", "verificationMethod": [ { "id": "did:nexus:agent-abc123#key-1", "type": "JsonWebKey2020", "publicKeyJwk": { ... } }, "service": [ { "id": "did:nexus:agent-abc123#a2a", "type": "A2AService", "serviceEndpoint": "https://agent.example.com/a2a" } ] } ``

**\*\*Benefício\*\*:** identidade verificável criptograficamente, sem precisar de CA central. --- # Capítulo 5 — Autorização por skill (scope-based) Cada skill tem seu próprio **\*\*scope\*\*** no token JWT. O agente que chama precisa ter o scope específico. ``json { "iss": "did:nexus:agent-abc123", "sub": "did:nexus:agent-def456", "aud": "did:nexus:agent-abc123", "exp": 1720636800, "scope": "skills:consultar-produto skills:enviar-mensagem", "permissions": [ { "skill": "consultar-produto", "rate\_limit": "100/h" }, { "skill": "enviar-mensagem", "rate\_limit": "50/h" } ] } ``

O agente **\*\*def456\*\*** pode chamar `consultar-produto` (até 100/h) e `enviar-mensagem` (até 50/h). Não pode chamar outras skills.

**\*\*Verificação no servidor\*\*:** ``python def check\_permission(token, skill\_id): if skill\_id not in token.scope: raise PermissionError(f"Skill {skill\_id} not in scope") # Verificar rate limit rate = get\_rate\_limit(token, skill\_id) if rate.exceeded(): raise RateLimitError(f"Rate limit exceeded for {skill\_id}") ``

--- # Capítulo 6 — Auditoria distribuída (audit logs) Todo request A2A é logado de forma **\*\*imutável\*\*** e **\*\*distribuída\*\***. **\*\*Estrutura do log\*\*:** ``json { "timestamp": "2026-07-10T12:00:00Z", "trace\_id": "trace-abc-123", "request\_id": "req-456", "caller\_did": "did:nexus:agent-def456", "callee\_did": "did:nexus:agent-abc123", "method": "tasks/send", "skill": "consultar-produto", "input\_hash": "sha256:...", "output\_hash": "sha256:...", "duration\_ms": 142, "status": "success", "auth\_token\_id": "tok-xyz" } ``

**\*\*Imutabilidade\*\*:** use append-only storage (S3, blockchain, ou append-only DB). **\*\*Distribuição\*\*:** envie para 3+ sistemas (S3, CloudWatch, Loki).

**\*\*Retenção\*\*:** 90 dias hot, 7 anos cold (compliance). --- # Capítulo 7 — Rate limiting entre agentes Rate limit é **\*\*obrigatório\*\*** em federação. Sem ele, 1 agente pode derrubar outro. **\*\*3 tipos de rate limit\*\*:**

- \*\*1. Por agente (identidade)\*\*** Cada agente tem um limite global (ex: 1000 req/h).
- \*\*2. Por skill\*\*** Cada skill tem limite específico (ex: 100/h).
- \*\*3. Por recurso\*\*** Limit por recurso específico (ex: "consultar-produto:produtoId=XPTO: 10/h").

**\*\*Implementação\*\*:** ``python class RateLimiter: def \_\_init\_\_(self, redis\_client): self.redis = redis\_client def check(self, agent\_did, skill\_id, max\_per\_hour=100): key = f"rate:{agent\_did}:{skill\_id}:{hour()}" current = self.redis.incr(key) if current > max\_per\_hour: raise RateLimitError(f"Limit {max\_per\_hour}/h exceeded") if current == 1: self.redis.expire(key, 3600) # 1h return current ``

--- # Capítulo 8 — Caso Nexus: 12 agentes em 4 plataformas **\*\*Cenário\*\*:** A Nexus tem 12 agentes A2A em 4 plataformas: - **\*\*Google Cloud\*\*** (3 agentes) - **\*\*AWS\*\*** (4 agentes) - **\*\*Azure\*\*** (3 agentes) - **\*\*On-premise\*\*** (2 agentes)

**\*\*Implementação Zero-Trust\*\*:**

- \*\*mTLS entre todos os pares\*\*:** cada agente tem certificado. CA interna Nexus assina.
- \*\*DIDs para identidade\*\*:** cada agente tem DID

único. 3. **\*\*JWT com scope granular\*\***: cada skill tem seu scope. Afiliados não veem skills administrativas. 4. **\*\*Rate limiting por agente\*\***: 1k req/h por agente. Spikes acima disso → fila. 5. **\*\*Audit logs em 3 destinos\*\***: S3, CloudWatch, Loki. 6. **\*\*Monitoring\*\***: Datadog + alertas P1/P2.

**\*\*Resultado\*\*** (12 meses): - **\*\*0 incidentes\*\*** de segurança - **\*\*99.97% uptime\*\*** - **\*\*< 200ms latência\*\*** entre agentes (mTLS overhead é mínimo)

--- # Capítulo 9 — Métricas de segurança 5 métricas para monitorar: **\*\*1. Taxa de autenticação falha\*\*** % de requests com auth inválida. **\*\*Meta: < 0.1%\*\***. **\*\*2. Latência do mTLS overhead\*\*** Tempo extra de TLS. **\*\*Meta: < 10ms\*\***. **\*\*3. Rate limit hits\*\*** Requests bloqueados por rate limit. **\*\*Meta: < 5%\*\***. **\*\*4. Skills chamadas vs skills disponíveis\*\*** Detectar uso anômalo. **\*\*Meta: ratio > 0.3\*\*** (caso contrário, agente está pedindo skills demais). **\*\*5. Audit log completeness\*\*** % de requests que foram logados. **\*\*Meta: 100%\*\***.

--- # Capítulo 10 — Quando o Zero-Trust atrapalha Zero-Trust adiciona **\*\*overhead\*\*** (latência + complexidade). Em alguns casos, não vale:

**\*\*Caso 1 — Agentes internos (mesma empresa)\*\*** Se os 2 agentes são seus e rodam na mesma VPC, mTLS é exagero. Use VPC internal + IAM. **\*\*Caso 2 — Throughput crítico (> 10k req/s)\*\*** mTLS adiciona ~5ms. Em alto volume, isso importa. Use TLS simples + WAF. **\*\*Caso 3 — Latência crítica (< 50ms)\*\*** Para real-time (videochamada, IoT), Zero-Trust adiciona latência demais. **\*\*Decisão\*\***: avalie **\*\*custo/benefício\*\*** antes de aplicar Zero-Trust everywhere.

--- # Epílogo — O futuro do trust entre agentes Em 2026, Zero-Trust entre agentes é **\*\*novidade\*\***. Em 2030, será **\*\*padrão\*\***.

**\*\*Tendências\*\***: 1. **\*\*DIDs everywhere\*\***: cada agente com identidade auto-soberana 2. **\*\*mTLS default\*\***: agente sem mTLS = agente inconfiável 3. **\*\*Audit logs com blockchain\*\***: imutabilidade distribuída 4. **\*\*AI-audited AI\*\***: agentes A2A que auditam outros agentes 5. **\*\*Zero-Trust as a service\*\***: vendedores oferecem Zero-Trust pronto **\*\*Recomendação para afiliados\*\***: comece pequeno. Implemente mTLS entre 2 agentes seus. Adicione JWT com scope. Documente skills. Faça audit log. Depois escale.

**\*\*E o mais importante\*\***: trate cada agente como se fosse um humano. Com permissões, com auditoria, com revisão. Porque no fim, **\*\*agentes são pessoas digitais\*\***.

--- # Apêndice — Checklist de implementação **\*\*Fase 1 — Mínimo viável (1 semana)\*\*** - [ ] Agent Card publicado em /.well-known/agent.json - [ ] mTLS configurado (certificado auto-assinado OK para começar) - [ ] Audit log em arquivo (append-only) - [ ] Rate limit simples (100/h por agente) **\*\*Fase 2 — Maturidade (1 mês)\*\*** - [ ] DIDs implementados - [ ] JWT com scope por skill - [ ] Audit log em 3 destinos - [ ] Monitoring com alertas - [ ] Documentação de skills clara **\*\*Fase 3 — Produção (3 meses)\*\*** - [ ] mTLS com CA real (não auto-assinado) - [ ] Token rotation automática - [ ] Rate limit distribuído (Redis) - [ ] Anomaly detection (ML) - [ ] Compliance audit (LGPD, SOC2) **\*\*Fase 4 — Excelência (6 meses)\*\*** - [ ] Zero-Trust as code (Terraform) - [ ] AI-audited AI - [ ] Blockchain audit log - [ ] Multi-region failover - [ ] Penetration testing trimestral

--- \*Fim da Apostila 30 · Federação Zero-Trust\* \*MMN\_IA Collective · 2026 · Licença: CC BY-SA 4.0\* **\*\*Zero-Trust não é paranoia. É a nova confiança.\*\***